COURSE MINI-PROJECT

24-788 Introduction to Deep Learning (Spring 2026)
DUE: May 1, 2026, 11:59 PM

March 17, 2026

Part I: Instructions

1 Overview

In this assignment, you will apply the concepts from the course to a real engineering or scientific problem. Specifically, you will be asked to implement (a) one baseline model from the course and (b) one model not explicitly covered in the course, to a prediction or generation task defined by a dataset of your choice. The goals of this project are to:

(1) Gain hands-on experience in the end-to-end pipeline of model selection, development, and training.

(2) Develop the ability to analyze model performance in context, connecting architectural choices to observed outcomes.

Your deliverable is a short report analyzing the relative performance of each model, formatted in the style of a conference submission. This report will be accompanied by a code repository that supports your reported results.

2 Group Structure and Scope

You may work individually or in groups of up to four. Team selections are due on **April 3**. The expected scope of the project scales with the size of your group:

Group Size	**Report Length**	**Minimum Scope**
1 person	3–4 pages	Baseline + 1 model variant
2 people	4–5 pages	Baseline + 2 contributions†
3 people	5–6 pages	Baseline + 3 contributions†
4 people	6–8 pages	Baseline + 4 contributions†

The page count provided includes figures and excludes references. Each "contribution" is either one additional model variant *or* one additional analysis beyond test-set metrics. Groups of 2+ may mix these freely as long as the total matches the number of additional members. For instance, a two-person group could (a) compare a baseline CNN model mentioned in the course (e.g., ResNet)

against two different variants (e.g. ResNeXt and ConvNeXt) or (b) compare a baseline CNN model against one CNN variant and additionally analyze how the learned representations produced by the body of the network differ. Both approaches satisfy the scope requirement.

> A model "variant" can be from a different architecture family than the baseline (e.g., a GNN model vs. a CNN model) if both architectures are reasonable choices for the problem, or a meaningful modification within a single architecture family. The key requirement is that you can reason **why** the variant should behave differently on your task. A model variant can generally be considered different enough from the baseline if it has been published as a methods contribution in a peer-reviewed venue. If in doubt about whether a model variant counts, please feel free to contact course staff.

3 Dataset Selection

You are free to choose a dataset from the curated list in Part II of this document, or propose your own for instructor approval.

3.1 Proposing Your Own Dataset

If you decide to use your own dataset, please submit a short dataset proposal by **April 3** with your team selection. This proposal will ask you to describe (a) what the data represents, (b) the approximate dataset size and structure, (c) what prediction task you intend to solve, and (d) what metrics you intend to use to evaluate performance as a paragraph or list of bullet points. Datasets must meet the following requirements:

- The dataset must be motivated by an engineering or scientific problem.
- The dataset should be small enough to train on in a reasonable timeframe (hours, not days) using available compute resources on Colab Pro.
- The dataset should be large enough to enable meaningful training. As a rule of thumb, this looks like a minimum of $\sim 10^3$ IID samples.

Although course staff can help guide the choice of dataset and high-level analysis decisions, they will be unable to provide low-level debugging assistance for proposed datasets.

4 Report Format

Your report should be formatted in the style of a conference submission. A template report format will be provided as `report_template.zip` with the structure and guiding questions that we expect the mini-project document to answer. Do not modify margins, font sizes, or spacing. Submit the final report as a PDF via Gradescope. This report will ask for an **Introduction** describing the context of the problem, a **Methods** section describing the baseline models and variant(s) that you chose, a **Results and Discussion** section evaluating the performance of your models in context, and a **Conclusion** summarizing the mini-project results. We encourage you to read this template prior to starting the mini-project.

Given the limited scope of the mini-project, the requirements of each section are not intended to be as extensive as a full conference paper, but the narrative function of each section is similar.

5 Code Submission

In addition to the report, submit your codebase as a GitHub repository (or equivalent). The repository should include:

1. **Your full codebase** used for data loading, training, and evaluation. There is no requirement on code structure or style, but readability is appreciated.
2. **A** `README.md` that explains:
   - How to set up the environment (dependencies, data download instructions).
   - How to reproduce the key results from your report.
3. **A** `reproduce_results` **script or notebook** that loads your trained model checkpoint(s) and regenerates the key figures and metrics reported in your paper. The teaching team should be able to run this without retraining from scratch, so please save your model checkpoint files.

6 Computing Resources

All students have access to **Google Colab Pro** for this project. Please keep the following constraints in mind when scoping your work:

- Colab Pro sessions have a **24-hour maximum runtime**. Your training jobs should complete well within this window. If a single training run takes more than a few hours, consider reducing model size or dataset scope.
- Datasets should be **under 10 GB**. Mount Google Drive or stream data as needed for storage.
- We recommend using an experiment tracking tool (e.g., Weights & Biases, MLflow, or TensorBoard) to log your training runs. This will save you from repeating work and will help you produce better figures for the report.

If you encounter compute limitations that prevent you from completing the project, contact the teaching team. We have a limited emergency compute budget available on a case-by-case basis.

7 Grading

Your project will be evaluated on the following criteria:

Criterion	Weight	Expectation
Problem Motivation	10%	Clear articulation of why the dataset and task matter from an engineering or scientific perspective.
Methods and Justification	25%	Correct description of baseline and variant(s). Clear hypothesis connecting architectural choices to data properties.
Experimental Design	15%	Appropriate choice of metrics and fair comparison between models.
Results and Analysis	30%	Quantitative results clearly reported. Discussion of why models performed as they did. For larger groups, additional experiments that are appropriate for the problem and analyzed clearly and thoughtfully.
Writing and Presentation	10%	Clear, concise writing. Effective use of figures and tables. Professional formatting.
Code and Reproducibility	5%	Repository is available and organized. Reproduction script or notebook runs and produces results consistent with the report.
Collaboration Statement	5%	A statement that clearly states what each group member was responsible for, and when and how AI tools were used within the mini-project. For solo students, this should only describe AI tool use.

Please note that we are not grading you on achieving state-of-the-art performance, and we care more about a thoughtful analysis of your results than the absolute accuracy numbers. However, we do expect best practices for training and evaluating models to be followed during the mini-project (e.g., training until convergence, choosing reasonable hyperparameters, etc.).

7.1 Bonus

Groups that exceed the scope requirement for their group size by meeting the requirements of the next tier up will receive a **+10 point bonus** (out of 100). For example, a solo student who delivers the scope of a two-person project (baseline + 2 contributions with a 4–5 page report) would receive 10 bonus points.

8 Academic Integrity

You may use any publicly available code, libraries, or pretrained components, but you must clearly cite them in your report and README. Using code from a published repository is fine, but copying results from a published paper and presenting them as your own is not. If you adapt an existing implementation, note what you changed and why. Unlike the homework assignments that focus specifically on implementation, you are allowed to use AI-based coding tools to assist with code

development. However, the analysis and written report **must represent your own work**. If course staff have clarifying questions about project content, we reserve the right to schedule a short meeting with project groups.

Part II: Suggested Datasets

The following datasets are pre-approved for the mini-project. You are not limited to this list, but these have been verified for public availability and compatibility with standard PyTorch workflows. Each entry includes a description of the engineering context, dataset format, suggested evaluation metrics, and practical tips.

A note on generative models. Any of the datasets below can also be used for a generation task using the generative baselines listed in the Appendix (VAE, GAN, Diffusion). For example, a student could train a VAE to generate plausible vorticity fields from the Kolmogorov Flow dataset, generate novel molecular graphs from QM9, or generate 3D point clouds from ShapeNet. If you choose this framing, your report should clearly define what "realistic generation" means for your domain and how you will evaluate it (e.g., comparing distributions of physical quantities, measuring diversity, or assessing physical plausibility of generated samples).

1 Kolmogorov Flow

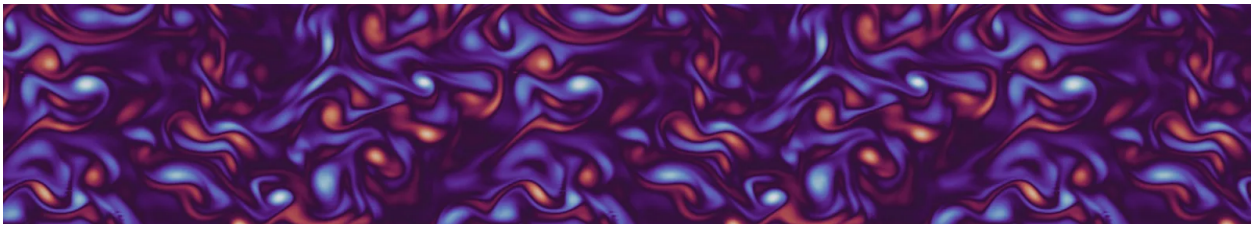

Figure 1: An example of Kolmogorov Flow with the vorticity field plotted.

1.1 Description

Kolmogorov flow is a special case of the Navier-Stokes equation described by a 2D laminar flow driven by a spatially periodic force. It is commonly used as a model system to study the transition to turbulence, flow instability, and energy cascades in fluids. This flow is characterized by its simplicity in analytical treatment and experimental realization and, as such, is a common benchmark for fluid solvers and neural PDE surrogates.

1.2 Dataset

This dataset consists of regularly gridded fluid simulations with 1 channel (vorticity) and a spatial resolution of 160×160. The data is saved at a resolution of $\Delta t = 0.1s$ for 200 timesteps, resulting in a rollout from $t = 0$ to $t = 20$ seconds. The initial conditions are sampled from a random truncated Fourier series with 5 modes, and the viscosity ν is set to 10^{-2} to simulate a Reynolds number of approximately 10^2. There are 256 samples in the dataset, each consisting of a different initial condition, totaling to around 5 GB of data.

The dataset can be found at https://huggingface.co/datasets/ayz2/temporal_pdes/tree/main/valid under the file name: KolmFlow_valid_256.h5. The dataset is formatted using h5py, and is organized as:

- Split [valid]
  - u : nodal values of the PDE solution, in shape [num_samples, temporal_resolution, spatial_resolution]
  - x : coordinates of the spatial domain, in shape [spatial_resolution]
  - t : timesteps of the PDE solution, in shape [temporal_resolution]

The task is to predict the forward evolution of the system in time, given an initial snapshot of the system.

1.3 Related Works

Suggested architectures could be FNO, Unet, or ViT-based models. Diffusion models, while more advanced, could also be a good choice. Some useful references are:

- https://arxiv.org/abs/2209.15616

- https://arxiv.org/abs/2010.08895
- https://arxiv.org/abs/2309.01745v3

In general, stable rollouts of Kolmogorov Flow are not tractable for neural PDE surrogates. With this dataset, most models start to decorrelate from the ground truth around 50 timesteps in. There are plenty of methods to try to improve this:

- https://arxiv.org/abs/2308.05732
- https://www.sciencedirect.com/science/article/pii/S0045782524006960
- https://www.sciencedirect.com/science/article/pii/S0045782525002622

1.4 Evaluations

- The standard evaluation for neural PDE surrogates is normalized RMSE, given a label u and prediction u_θ:

$$NRMSE(\mathbf{u}, \mathbf{u}_\theta) = \frac{1}{n_t} \sum_{t=1}^{n_t} \frac{||\mathbf{u}_\theta(t) - \mathbf{u}(t)||_2}{||\mathbf{u}(t)||_2 + \epsilon}, \quad ||\mathbf{u}||_2 = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} |\mathbf{u}(i,j)|^2} \tag{1}$$

- It is important to understand the one-step error of the surrogate model ($L(f_\theta(u^t), u^{t+1})$) as well as the rollout error of the model ($\frac{1}{T}\sum_t^T L(u_\theta^t, u^t)$). The rollout error is evaluated autoregressively.
- Kolmogorov Flow exhibits a predictable energy cascade of its log turbulent kinetic energy (log-TKE). If you are able to calculate this, it would be good to see if the neural PDE surrogate is able to reproduce the correct flow statistics.

1.5 Tips

- H5 datasets are organized as a dictionary (access via dataset["key"]). You can view the keys at any level using (dataset.keys()), and load the tensor from the disk using (data = torch.tensor(dataset["key"])).
- Do not attempt to load the entire dataset as a tensor. Instead, access individual samples in the dataset as you need them (data_i = torch.tensor(dataset["valid"]["u"][i]))
- Split the 256 validation samples into a train/val set. The validation set is provided to train on since it is smaller than the actual training set (20GB). The full training set exceeds the 10 GB Colab Pro guideline, so we recommend using the validation split unless you have access to additional compute.
- Convolutional architectures tend to work well since this dataset is regularly gridded.

2 Cylinder Flow

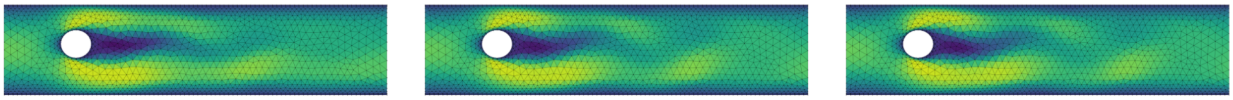

Figure 2: Example of one sample of the dataset over time. Velocity in the x-direction is plotted.

2.1 Description

Cylinder flow describes laminar flow in a 2D channel with a cylindrical obstacle. Since it is simple to study experimentally, it is a well-understood system, making it a common benchmark for numerical methods and neural PDE surrogates. The behavior is characterized by the wake of the cylinder, which can exhibit vortex shedding at certain Reynolds numbers. Additionally, the system is described by the inlet velocity, cylinder radius, and cylinder position, which all affect the flow.

2.2 Dataset

The original data are generated using COMSOL, with 25 timesteps from $t = 0$ to $t = 6$ on a 2D spatial domain ($x = [0, 1.6], y = [0, 0.4]$). Each sample is varied in the position and radius of the cylinder, as well as in the inlet velocity of the fluid. This results in samples with Reynolds numbers of 100-1500 and includes Karman vortex streets. The dataset is irregularly gridded on a mesh, resulting in a different number of points for each sample. Additionally, each point contains velocity and pressure information, resulting in 3 channels (x,y-velocity, pressure).

The dataset can be found at https://huggingface.co/datasets/ayz2/ldm_pdes/tree/main under the file name: cylinder_flow_captioned.zip. The dataset is formatted using h5py, and is organized as:

- `dataset.h5` (keys: `'0'`, `'1'`, ... etc.)
  - `'0'` (keys: `'cells'`, `'mesh_pos'`, `'metadata'`, `'node_type'`, `'pressure'`, `'u'`, `'v'`)
    - `'cells'`: shape `(num_edges, 3)`. Defines connectivity in triangular mesh. Only used for plotting.
    - `'mesh_pos'`: shape `(num_nodes, 2)`. Defines the position of each node in the mesh.
    - `'node_type'`: shape `(num_nodes, 1)`. Defines type of each node (0=fluid, 4=inlet, 5=outlet, 6=boundaries/walls).
    - `'pressure'`: shape `(num_timesteps, num_nodes, 1)`. Defines pressure at each timestep for all mesh points.
    - `'u'`: shape `(num_timesteps, num_nodes)`. Defines x-component of velocity at each timestep for all mesh points.
    - `'v'`: shape `(num_timesteps, num_nodes)`. Defines y-component of velocity at each timestep for all mesh points.
    - `'metadata'` (keys: `'center'`, `'domain_x'`, `'domain_y'`, `'radius'`, `'reynolds_number'`, `'t_end'`, `'u_inlet'`, `'v_inlet'`)
      - `'center'`: shape `(2,)`. Extracted center of cylinder, in meters.
      - `'domain_x'`: shape `(2,)`. Bounds of x in the domain, in meters.

    - `'domain_y'`: shape `(2,)`. Bounds of y in the domain, in meters.
    - `'radius'`: shape `()`. Extracted radius of cylinder, in meters.
    - `'reynolds_number'`: shape `()`. Extracted Reynolds number of simulation.
    - `'t_end'`: shape `()`. Final time of simulation.
    - `'u_inlet'`: shape `()`. x-component of velocity at the inlet.
    - `'v_inlet'`: shape `()`. y-component of velocity at the inlet.
  - `'1'`, `'2'`, ...etc.

The task is to predict the forward evolution of the system in time, given an initial snapshot of the system.

2.3 Related Works

Since this is an irregular grid, suggested architectures would be GNN, Transformer, or Aggregation-based models. Some useful references are:

- https://arxiv.org/abs/2010.03409
- https://arxiv.org/abs/2402.02366
- https://arxiv.org/abs/2505.12944
- https://arxiv.org/abs/2502.09692

Aggregation-based models work by interpolating irregular data onto a regular grid, then applying standard convolution. After convolution, the features are scattered onto the output grid as the prediction. The interpolation/scattering is usually learnable.

2.4 Evaluations

- The standard evaluation for neural PDE surrogates is normalized RMSE, given a label u and prediction u_θ:

$$NRMSE(\mathbf{u}, \mathbf{u}_\theta) = \frac{1}{n_t} \sum_{t=1}^{n_t} \frac{||\mathbf{u}_\theta(t) - \mathbf{u}(t)||_2}{||\mathbf{u}(t)||_2 + \epsilon}, \quad ||\mathbf{u}||_2 = \sqrt{\frac{1}{n_x n_y} \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} |\mathbf{u}(i,j)|^2} \tag{2}$$

- It is important to understand the one-step error of the surrogate model ($L(f_\theta(u^t), u^{t+1})$) as well as the rollout error of the model ($\frac{1}{T}\sum_t^T L(u_\theta^t, u^t)$). The rollout error is evaluated autoregressively.

2.5 Tips

- H5 datasets are organized as a dictionary (access via dataset["key"]). You can view the keys at any level using (dataset.keys()), and load the tensor from the disk using (data = torch.tensor(dataset["key"])).
- In general, training with irregular data is more challenging than regularly-gridded problems. Batching is also a concern, therefore training with a batch size of 1 and using gradient accumulation is a common strategy.

3 Nanofiltration Rejection Prediction

3.1 Description

Nanofiltration (NF) is a membrane-based separation technology widely used in chemical manufacturing, pharmaceuticals, and water treatment. In NF systems, solvents and solutes pass through a semi-permeable membrane under pressure, and the membrane selectively rejects certain molecules based on size, charge, and chemical affinity. Predicting solute rejection is a critical step in designing efficient separation processes, as it determines the feasibility of a given membrane for a target application. The relationship between rejection and the underlying system parameters (molecular structure, membrane properties, solvent, operating conditions) is complex and nonlinear, making it a natural candidate for data-driven modeling.

3.2 Dataset

The NF-10K dataset contains approximately 9,921 nanofiltration measurements collected from literature studies, covering 1,089 unique organic solutes across different solvents, membranes, and operating conditions. Each entry corresponds to a solute–membrane–solvent system. The dataset is tabular with the following columns:

- `smiles`: SMILES representation of the solute molecule
- `solvent`: solvent used in the experiment
- `membrane`: membrane type
- `pressure`: operating pressure
- `temperature`: operating temperature
- `rejection`: experimental rejection (%), the prediction target

The dataset can be found at https://osndb.kaust.edu.sa/en-US/#/datasets under the name NF-10K.

The task is to predict rejection (%) for a given solute–membrane–solvent system. Molecular structures are provided as SMILES strings and must be converted to numerical features (e.g., molecular fingerprints, molecular descriptors, or molecular graph representations) before use.

3.3 Related Works

Suggested architectures could be GNN-based models (GAT, MPNN), Transformer-based molecular models, or descriptor-based deep learning approaches (MLP on molecular fingerprints). Some useful references are:

- https://www.nature.com/articles/s41560-024-01668-7
- https://www.nature.com/articles/s42256-022-00447-x
- https://arxiv.org/abs/1810.00826
- https://arxiv.org/abs/2003.00982

Graph neural networks treat molecules as graphs where atoms are nodes and bonds are edges. Message passing operations allow the network to learn representations of chemical structure that influence molecular behavior.

3.4 Evaluations

- The primary evaluation metrics are Root Mean Squared Error (RMSE) and Coefficient of Determination (R^2):

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{3}$$

  where y_i is the true rejection value and $\hat{y}_i$ is the predicted value.

- The benchmark performance reported in the first reference study on the dataset is approximately RMSE $= 0.122$ and $R^2 = 0.89$. It may be useful to use this performance as a point of comparison.

3.5 Tips

- SMILES strings must be converted to features using tools such as RDKit. Molecular fingerprints are the simplest featurization and pair naturally with an MLP baseline.

- When working with molecular graphs, libraries such as PyTorch Geometric or DGL can simplify data loading and model implementation.

- Feature normalization and careful dataset splitting are important to prevent data leakage, particularly since multiple measurements may share the same solute or membrane.

4 QM9 Molecular Properties

4.1 Description

Computational chemistry relies on quantum-mechanical simulations to predict molecular properties such as orbital energies, heat capacity, and atomization energy. These simulations, typically based on density functional theory (DFT), are accurate but computationally expensive, often requiring hours per molecule. Data-driven surrogate models that predict molecular properties directly from molecular structure can accelerate materials screening, drug discovery, and chemical design by orders of magnitude. The QM9 dataset is one of the most widely used benchmarks for evaluating such models.

4.2 Dataset

QM9 contains approximately 130,000 small organic molecules, each with up to 9 heavy atoms (C, N, O, F) and associated hydrogen atoms. For each molecule, the dataset provides 19 regression targets computed via DFT, including properties such as HOMO/LUMO energies, dipole moment, heat capacity, and atomization energy. Each molecule is represented as a graph where atoms are nodes and bonds are edges, with full 3D coordinate information for the equilibrium geometry.

The dataset is available directly through PyTorch Geometric:

```
from torch_geometric.datasets import QM9
dataset = QM9(root='./data/QM9')
```

Each `Data` object contains:

- `x`: node (atom) features, shape `[num_atoms, 11]`
- `edge_index`: bond connectivity, shape `[2, num_bonds]`
- `edge_attr`: bond features, shape `[num_bonds, 4]`
- `pos`: 3D atom coordinates, shape `[num_atoms, 3]`
- `y`: target properties, shape `[1, 19]`

The task is to predict one or more molecular properties from the molecular graph. Students should select a specific target property (e.g., HOMO-LUMO gap, atomization energy) and report results on that target.

4.3 Related Works

Suggested baseline architectures include GCN or a general message-passing GNN. Variants could include models that incorporate 3D geometric information, such as SchNet, DimeNet, or equivariant GNNs. Some useful references are:

- https://arxiv.org/abs/1704.01212
- https://arxiv.org/abs/1706.08566
- https://arxiv.org/abs/2003.03123
- https://arxiv.org/abs/1703.00564

4.4 Evaluations

- The standard evaluation metric is Mean Absolute Error (MAE) on the held-out test set for each target property:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i| \tag{4}$$

- A common benchmark target is the HOMO-LUMO gap ($\Delta\epsilon$, target index 4), which has practical relevance to photovoltaics and organic electronics. State-of-the-art models achieve MAEs below 50 meV on this target.
- Students should report MAE in the original physical units for their chosen target property.

4.5 Tips

- PyTorch Geometric handles downloading and preprocessing automatically. No additional dependencies are required unless you want to process raw SDF files (which requires `rdkit`).
- The dataset is small in storage (∼300 MB) and molecules are small (up to ∼29 atoms), so training is fast even on Colab.
- Target properties span very different scales. Normalize your targets before training (e.g., subtract mean, divide by standard deviation) and convert back for evaluation.
- A standard train/validation/test split is 110k/10k/10k. You can use a random split or follow the split from the SchNet paper.

5 ShapeNet Part Segmentation

5.1 Description

Understanding 3D shape structure is fundamental to computer-aided design, robotics, and manufacturing. Part segmentation, the task of labeling each point in a 3D shape with its semantic part (e.g., the wing, fuselage, and engine of an airplane), is a core capability for applications such as automated assembly, quality inspection, and shape retrieval. The ShapeNet dataset provides a large-scale collection of 3D CAD models annotated with fine-grained part labels, making it the standard benchmark for point cloud segmentation methods.

5.2 Dataset

The ShapeNet part segmentation dataset contains approximately 16,880 3D shapes across 16 object categories (e.g., airplane, chair, car, table, guitar, lamp), annotated with a total of 50 part labels. Each shape is represented as a point cloud with 2,048 points, where each point has 3D coordinates (x, y, z) and a normal vector (n_x, n_y, n_z). Each point is labeled with one of the part categories specific to its object class.

The dataset is available directly through PyTorch Geometric:

```
from torch_geometric.datasets import ShapeNet
dataset = ShapeNet(root='./data/ShapeNet', categories=['Airplane'])
```

Each `Data` object contains:

- `pos`: point coordinates, shape `[2048, 3]`
- `x`: point normals, shape `[2048, 3]`
- `y`: per-point part labels, shape `[2048]`
- `category`: integer index for the object category

The task is per-point part segmentation: given a point cloud, predict the part label for each point. Students may choose to work with all 16 categories or a subset.

5.3 Related Works

The natural baseline is PointNet, which processes point clouds directly. Suggested variants include PointNet++ (hierarchical point set learning), DGCNN (dynamic graph CNN), or point-based Transformer architectures. Some useful references are:

- https://arxiv.org/abs/1612.00593
- https://arxiv.org/abs/1706.02413
- https://arxiv.org/abs/1801.07829
- https://arxiv.org/abs/2012.09164

5.4 Evaluations

- The standard evaluation metric is mean Intersection over Union (mIoU), computed per-category and then averaged:

$$\mathrm{IoU}_c = \frac{|\hat{S}_c \cap S_c|}{|\hat{S}_c \cup S_c|}, \quad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c \tag{5}$$

  where S_c and $\hat{S}_c$ are the ground truth and predicted point sets for part c, and C is the number of part labels.

- Instance-averaged mIoU (average the mIoU of each shape, then average across shapes) is the standard protocol for ShapeNet.

- Per-point classification accuracy is a useful secondary metric but can be misleading when part sizes are imbalanced.

5.5 Tips

- PyTorch Geometric handles downloading and preprocessing. You can load all categories or a subset using the `categories` argument.

- The dataset is relatively small ($\sim$1 GB) and trains quickly on Colab.

- Part labels are category-specific (e.g., airplanes have 4 parts, chairs have 4 parts, but the label sets are different). If training across all categories, you will need to handle this in your model output layer.

- For PointNet-based models, data augmentation via random rotation and jittering of point coordinates is standard practice and improves generalization.

6 NASA C-MAPSS Turbofan Engine Degradation

6.1 Description

Predicting when a machine component will fail is a central problem in prognostics and health management (PHM). In safety-critical systems such as aircraft engines, accurate remaining useful life (RUL) estimation enables condition-based maintenance, reducing both unplanned downtime and unnecessary part replacements. The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset is the most widely used benchmark in the prognostics community, providing simulated run-to-failure trajectories from turbofan engines with realistic sensor noise and operating condition variability.

6.2 Dataset

The C-MAPSS dataset contains four subsets (FD001–FD004) of increasing complexity. We recommend starting with **FD001**, which contains 100 training engine trajectories and 100 test trajectories under a single operating condition with a single fault mode (high-pressure compressor degradation).

Each trajectory is a multivariate time series recorded from engine start until failure. At each time step, the following are recorded:

- `unit`: engine unit number
- `cycle`: time step index within the trajectory
- 3 operational setting variables
- 21 sensor measurements (temperatures, pressures, speeds, etc.)

The data is provided as space-delimited text files. For the training set, each trajectory runs until failure. For the test set, trajectories are truncated at some point before failure, and the true RUL values are provided separately.

The dataset can be downloaded from the NASA Prognostics Data Repository: https://phm-datasets.s3.amazonaws.com/NASA/6.+Turbofan+Engine+Degradation+Simulation+Data+Set.zip

The task is to predict the remaining useful life (number of cycles until failure) given the sensor readings up to the current time step.

6.3 Related Works

The natural baseline is an LSTM or RNN operating on the sensor time series. Transformer-based models and temporal convolutional networks are natural variants. Some useful references are:

- https://arxiv.org/abs/1709.01073
- https://doi.org/10.1016/j.ress.2017.11.021
- https://doi.org/10.3390/s18051991

6.4 Evaluations

- The standard evaluation metric is Root Mean Squared Error (RMSE) on the predicted RUL values:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\text{RUL}_i - \widehat{\text{RUL}}_i)^2} \tag{6}$$

- An asymmetric scoring function is also standard in the PHM community, which penalizes late predictions (predicting failure after it occurs) more heavily than early predictions. The scoring function is:

$$s_i = \begin{cases} e^{-d_i/13} - 1 & \text{if } d_i < 0 \\ e^{d_i/10} - 1 & \text{if } d_i \geq 0 \end{cases}, \quad d_i = \widehat{\text{RUL}}_i - \text{RUL}_i \tag{7}$$

- A common practice is to clip the target RUL at a maximum value (e.g., 125 cycles), since early in an engine's life the degradation signal is negligible and the exact RUL is not meaningfully predictable.

6.5 Tips

- The text files can be loaded directly with `pandas.read_csv` using whitespace as a delimiter. Column headers are not included in the files, so you will need to assign them manually.
- Some sensor columns are nearly constant and carry little information. Exploratory analysis or feature selection before modeling is recommended.
- Normalize sensor readings across the dataset. Operating condition variables should be treated carefully—in FD001 they are constant, but in FD002–FD004 they vary and significantly affect sensor readings.
- For sequence models, you will need to decide on a fixed input window length. A sliding window of the most recent W time steps is standard; $W = 30$–50 is a common range.
- Start with FD001. If your project scope allows, comparing performance across FD001–FD004 (increasing operating conditions and fault modes) is a strong additional analysis.

Appendix: Models Covered in This Course

The following architectures were explicitly covered in lecture and are eligible as **baseline** models. Your model variant should not appear on this list. It must be grounded in a separate published, peer-reviewed paper, even if it belongs to the same model family as your baseline.

Data Modality	**Models Covered**
Tabular / Generic	Multilayer Perceptron (MLP)
Images / Grid Data	Convolutional Neural Network (CNN) LeNet, AlexNet, VGG, GoogLeNet/Inception, ResNet Fully Convolutional Network (segmentation) U-Net (segmentation) YOLOv1 (object detection)
Sequences / Time Series	Recurrent Neural Network (RNN) Long Short-Term Memory (LSTM) Gated Recurrent Unit (GRU) Transformer (Vaswani et al.)
Graphs / Point Clouds	PointNet Graph Convolutional Network (GCN)
Self-Supervised Learning	SimCLR Masked Autoencoder BERT
Generation	Variational Autoencoder (VAE) Generative Adversarial Network (GAN) Diffusion

If a model appears in this table, you may use it as your baseline but **not** as your variant. For example, ResNet is on this list, so it can serve as a baseline for image tasks. A variant like ResNeXt, ConvNeXt, or a Dense Transformer would be acceptable because they were not covered in lecture and each has a corresponding peer-reviewed publication. When in doubt, check with course staff.